Timed Traffic Controller

Observe a request, decide by time, show the state

The lamps are both the circuit's output and its live explanation.

Lesson	011 — deterministic behavior	**Time**	150–210 minutes
Prerequisites	Lessons 001–010	**Board**	Arduino Mega 2560
Evidence	nine LEDs, button, named lamp test points	**Status**	host verified; Mega compiled; bench open
Safety	E1: USB-powered, resistor-limited LEDs only		

1 Mission and evidence chain

Build the nonblocking controller that lesson 012 will compose into a tabletop junction. Its evidence chain is:

D22 press → debounced event → latched request → timed phase → D30–D37 lamps.

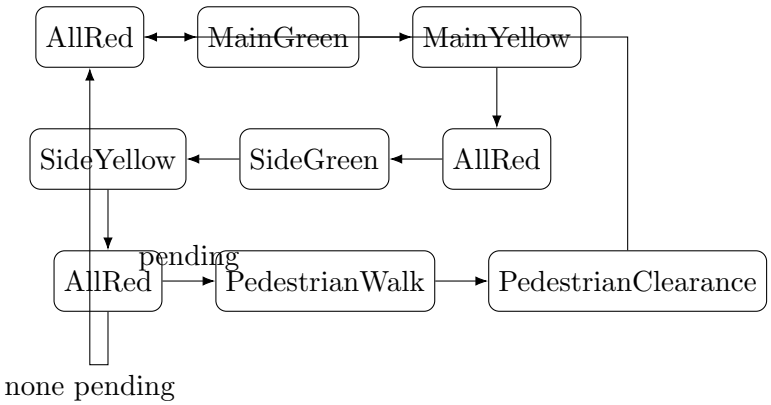
The D13 ready LED separately reports successful resource acquisition. It does not claim that the signal policy is correct. The eight signal LEDs expose the policy while it runs; a meter at a named lamp pin proves the electrical level.

By the end, you can:

- distinguish elapsed-time decisions from blocking delays;
- supply all time and input events explicitly to `TrafficJunction`;
- explain request latching and the release-gated `Button`;
- predict every phase and deadline from a recorded trace;
- interpret ready, signal, and meter evidence without relying on Serial;
- inject an unhealthy-circuit input and explain the latched fault.

Safety checkpoint. Disconnect USB before wiring. Every LED needs its own 330Ω or larger series resistor. Verify LED polarity and the breadboard's internal connections unpowered. Stop for heat, odor, resets, unstable USB power, or a signal combination that differs from the truth table. This is a low-voltage learning model, not a road-signal controller.

2 The state machine is the design



The first `update()` anchors startup `AllRed`; it does not borrow time from construction or from Arduino's global clock. Every later transition occurs when the supplied `TimePoint` reaches the exact deadline. A press event becomes a pending request. Startup may serve it immediately after startup all-red; otherwise the controller retains it until the next safe pedestrian window after the side-road sequence.

Phase	Lamps that are on	Invariant
AllRed	main red, side red, pedestrian stop	no vehicle green
MainGreen	main green, side red, pedestrian stop	one vehicle green
MainYellow	main yellow, side red, pedestrian stop	no vehicle green
SideGreen	main red, side green, pedestrian stop	one vehicle green
SideYellow	main red, side yellow, pedestrian stop	no vehicle green
PedestrianWalk	both vehicle reds, pedestrian walk	vehicles held red
PedestrianClearance	both vehicle reds, pedestrian stop	vehicles held red
Fault	both vehicle reds, pedestrian stop	latched until explicit reset

3 Orientation drawing

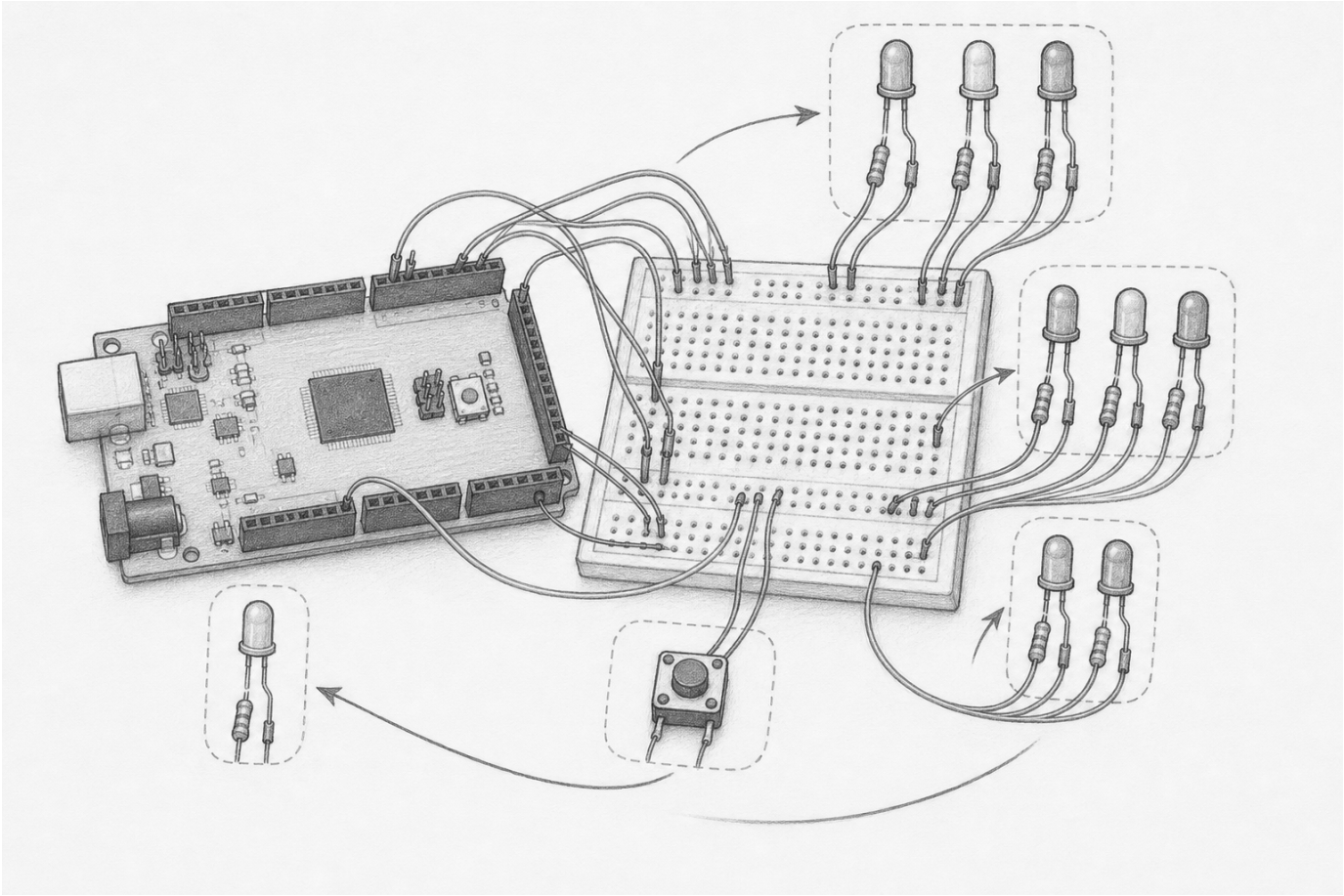


Figure 1: Pencil orientation showing two vehicle lamp groups, a pedestrian lamp pair, request button, ready indicator, Mega, and breadboard. This is not an authoritative wiring diagram; build only from the literal connection table below.

4 Parts and literal wiring

Quantity	Part	Requirement
1	Mega 2560	USB powered
4	red LEDs	identified polarity and forward-voltage range
2	yellow LEDs	identified polarity and forward-voltage range
3	green LEDs	two vehicle, one pedestrian/ready as assigned below
9	resistors	330Ω or larger, one per LED
1	momentary button	normally open, used with internal pull-up
1	multimeter	DC voltage and unpowered continuity

Every LED path: named Mega pin → 330 Ω → LED anode; LED cathode → GND. D22 connects through the normally open button to GND; `Pull::Up` supplies its idle level.

Function	Pin	Circuit-native observation
main red / yellow / green	D30 / D31 / D32	main signal head
side red / yellow / green	D33 / D34 / D35	side signal head
pedestrian walk / stop	D36 / D37	pedestrian signal head
request button	D22 to GND	press event; pending state appears later as walk
resource ready	D13	on only after all objects initialize
electrical test point	selected lamp pin to GND	near 0 V off; near 5 V on

With a conservative 2.0 V LED forward voltage, a 330Ω resistor permits about $(5 - 2)/330 = 9.1$ mA. The normal truth table lights four LEDs at most when D13 ready is included. Record actual forward voltages and currents; do not substitute assumed values for bench evidence.

5 Read the public model

```
adk::TrafficConfig config;
adk::TrafficJunction traffic (config);

traffic.initialize ();
traffic.update (now, adk::TrafficInput (requestEvent, circuitHealthy));

const adk::TrafficSnapshot decision = traffic.snapshot ();

traffic.reset    ();
traffic.shutdown ();
```

`TrafficConfig` names each duration: `startupAllRed`, `vehicleAllRed`, `mainGreen`, `mainYellow`, `sideGreen`, `sideYellow`, `pedestrianWalk`, and `pedestrianClearance`. Defaults are 1,000; 500; 5,000; 1,500; 4,000; 1,500; 5,000; and 2,000 ms respectively.

`TrafficSnapshot` exposes phase and status, output signals, phase start and next deadline, request state, change and acceptance events, deadline presence, and transition count:

```
phase, status, signals,
phaseSince, nextDeadline,
pedestrianRequestPending,
phaseChanged, requestAccepted, hasDeadline,
transitionCount
```

Observation does not consume an event. Supplying the same configuration, timestamps, and input events produces the same snapshots. Repeated `initialize()` is idempotent: it preserves the active phase, timing, pending request, or fault latch. `reset()` explicitly starts a new run in startup `AllRed`; it requires an initialized object. `shutdown()` is idempotent, leaves an inert all-red snapshot, and makes later `update()` calls report `NotInitialized`.

The canonical sketch uses the same high-level language as the circuit:

```
const adk::TrafficInput observation = observeRequest (now);
const adk::Status       decision    = decideSignals  (now, observation);
const bool              signalsShown = showSignals   (traffic.snapshot ().signals);

if (!signalsShown)
{
    stopSafely ();
}

if (!decision.ok ())
{
    ready.off ();
}
```

There is no `delay()`. `loop()` remains free to debounce the button and actuate the current signal during every phase. The complete, canonical source is Lesson011TimedTraffic.

6 CLI experiment

1. With USB disconnected, check every LED path and confirm no short from 5 V to GND.
2. Predict the startup lamps and the first four deadlines.
3. Build with `make arduino-Lesson011TimedTraffic`.
4. Upload with `make upload-Lesson011TimedTraffic PORT=/dev/ttyACM0`.
5. Observe D13 ready separately from the eight policy lamps.
6. Press D22 briefly during `MainGreen`. Continue watching: the request is retained, not served by interrupting a vehicle phase.
7. Measure D32 to GND during main green and all-red; record both voltages.
8. Optional supporting evidence: `make serial-log PORT=/dev/ttyACM0`. The sketch needs no Serial log to prove its state.

> **Predict.** A request during main green cannot immediately turn on walk. **Observe.** Main completes, side completes, both vehicle reds appear, then walk appears. **Interpret.** The event was latched and served only at a designed boundary. This evidence does not by itself prove the pin voltages or resource cleanup.

7 Deterministic trace worksheet

Use shortened test durations on the host, not on an unexplained bench circuit: startup all-red 100 ms, vehicle all-red 50 ms, main green 300 ms, main yellow 100 ms, side green 200 ms, side yellow 100 ms, walk 250 ms, clearance 150 ms.

Time	Input event	Predicted phase	Snapshot evidence
0 ms	none	_____	deadline _____
100 ms	none	_____	transition count _____
220 ms	request	_____	pending _____
400 ms	none	_____	lamps _____
500 ms	none	_____	deadline _____
700 ms	none	_____	lamps _____
800 ms	none	_____	pending _____
850 ms	none	_____	lamps _____

Replay the table twice. Compare every snapshot field, not merely the final phase. Then shift the request to the startup all-red interval and explain why the service order changes without becoming nondeterministic.

8 Faults, diagnosis, and safe evidence

Passing `TrafficInput(false, false)` enters `Fault`, selects both vehicle reds and pedestrian stop, returns a failure status, and latches that condition. Repeated `initialize()` deliberately preserves this latch; only `reset()` begins a new run. The adapter presents the all-red fault signals and turns D13 off. It treats an output-write failure more conservatively: it shuts down the state engine and every `MonoLed`, making each owned output high impedance.

Observation	Likely layer	Next evidence
D13 never lights	acquisition or wiring	unpowered continuity; inspect pin claims
D13 lights, one head dark	LED path or actuation	measure the named pin and resistor
request never served	input wiring or event timing	observe D22 and button debounce
walk interrupts green	policy defect	stop; preserve timestamp/input trace
both greens appear	invariant violation or wiring	remove USB immediately
lamps freeze but loop runs	timestamp/configuration	inspect deadlines with host trace
all lamps extinguish after failure	application safe shutdown	measure high impedance

Resource evidence is D13 turning on only after ten pin owners and the state engine initialize. **Policy evidence** is the truth-table lamp pattern. **Safe-state evidence** after an application failure is the measured high-impedance output, not merely a dark LED. These are three separate claims.

9 Exercises

1. Draw the exact phase reached at every default deadline for one complete cycle with no request.
2. Record two request events during one cycle. Explain why one pending bit is sufficient for this lesson and what information it intentionally does not count.
3. Add a circuit-native pending indicator in a host-only design. State its extra pin and current cost before changing hardware.
4. Inject decreasing time, a zero duration, and an unhealthy circuit into deterministic tests. Predict the returned `Status`.
5. Explain why a blinking pedestrian-clearance display belongs in a presentation adapter rather than hidden timing inside the engine.
6. Propose a power-on lamp test that does not weaken the all-red invariant.

10 Bench acceptance record

Do not mark this lesson hardware verified until every blank is measured.

Record	Observation
Board and supply	Mega 2560 revision _____; USB supply _____
Resistors	D13, D30–D37 measured values _____
Startup	D13 sequence _____; initial signal pattern _____
Normal cycle	observed phase order and durations _____
Request	press phase _____; served phase/time _____
Electrical	D32 on _____ V; off _____ V; reference GND _____
Reset	observed startup all-red behavior _____
Shutdown	selected output high-impedance measurement _____
Current	maximum simultaneous LED current _____ mA
Tools	meter/model _____; CLI/core versions _____
Result	☐ pass ☐ deviation attached date/reviewer _____

Current status: host verified and Mega 2560 firmware compiled; physical bench acceptance remains open until the completed record is published.